Experiment No : 03

Aim :

Learn basics of maplotlib library and its use as visualisation tool in data science pipeline and learn about line plots and scatter plots.

Theory :

Matplotlib is a multi-platform data visualization library built on NumPy arrays, and designed to work with the broader SciPy stack.

It was conceived by John Hunter in 2002, originally as a patch to IPython for enabling interactive MATLAB-style plotting via gnuplot from the IPython command line.

One of Matplotlib's most important features is its ability to play well with many operating systems and graphics backends. Matplotlib supports dozens of backends and output types, which means you can count on it to work regardless of which operating system you are using or which output format you wish.

This cross-platform, everything-to-everyone approach has been one of the great strengths of Matplotlib. It has led to a large user base, which in turn has led to an active developer base and Matplotlib's powerful tools and ubiquity within the scientific Python world.

In recent years, however, the interface and style of Matplotlib have begun to show their age. Newer tools like ggplot and ggvis in the R language, along with web visualization toolkits based on D3js and HTML5 canvas, often make Matplotlib feel clunky and old-fashioned.

Performance :

[Students need to execute eaxh and every cell in this section and note the output of the same. Once done they have to answer Questions mentioned in review section]

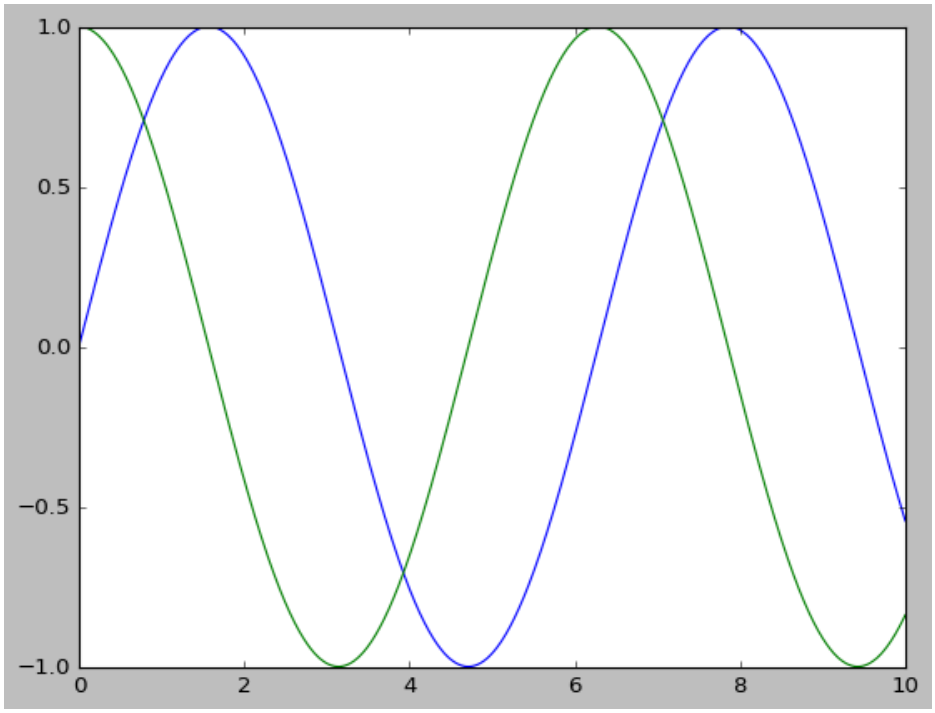
```
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.__version__
```

```
'3.10.0'
```

```
#setting ploting style
plt.style.use('classic')
```

```
%matplotlib inline
import numpy as np
x = np.linspace(0, 10, 1000)

fig = plt.figure()
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x));
```



Saving Figures to File

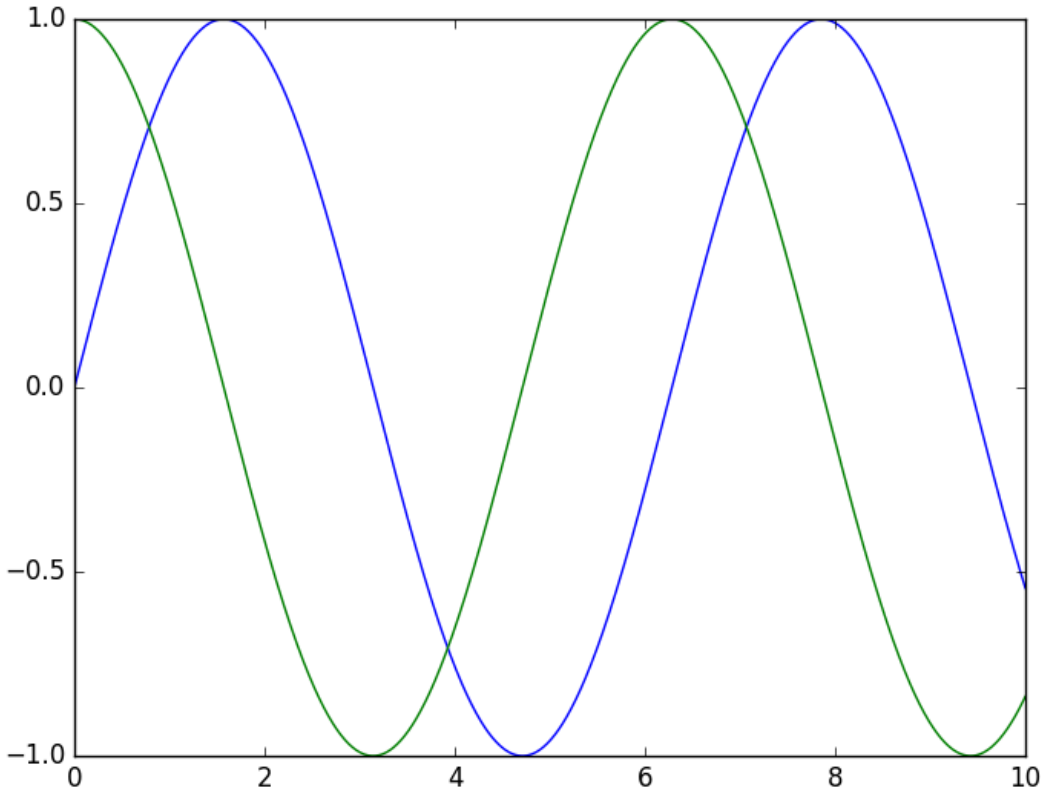
One nice feature of Matplotlib is the ability to save figures in a wide variety of formats. Saving a figure can be done using the `savefig()` command. For example, to save the previous figure as a PNG file, you can run this:

```
#Save above plot with file named my_figure.png in Current Working Directory
fig.savefig('my_figure.png')
```

```
# Cross check the folder content
!ls -lh my_figure.png
```

```
-rw-r--r-- 1 root root 45K Feb 25 06:50 my_figure.png
```

```
# use Image package in python to check contents of png file in cwd
from IPython.display import Image
Image('my_figure.png')
```




```
#checking supported file types by matplotlib
fig.canvas.get_supported_filetypes()
```

```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

Matplotlib API Interfaces

A potentially confusing feature of Matplotlib is its dual interfaces: a convenient MATLAB-style state-based interface, and a more powerful object-oriented interface. This section we will review them breifly.

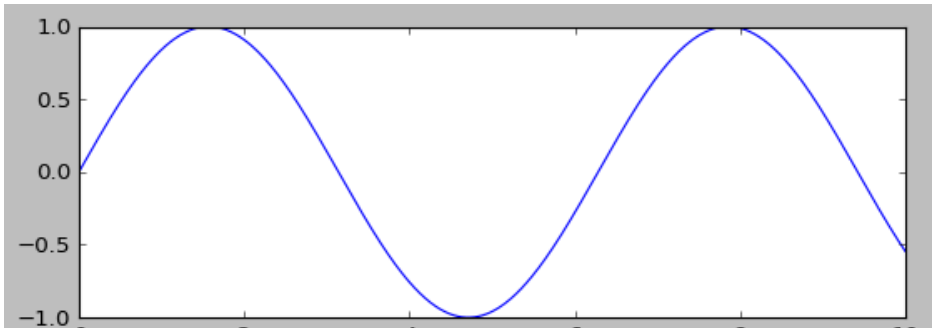
MATLAB-style Interface

Matplotlib was originally written as a Python alternative for MATLAB users, and much of its syntax reflects that fact. The MATLAB-style tools are contained in the pyplot (`plt`) interface. See the example inthe code cell below.

```
plt.figure()  # create a plot figure

# create the first of two panels and set current axis
plt.subplot(2, 1, 1) # (rows, columns, panel number)
plt.plot(x, np.sin(x))

# create the second panel and set current axis
plt.subplot(2, 1, 2)
plt.plot(x, np.cos(x));
```



This interface is *stateful*: It keeps track of the "current" figure and axes, which are where all `plt` commands are applied.

You can get a reference to these using the `plt.gcf()` (get current figure) and `plt.gca()` (get current axes) routines.

While this stateful interface is fast and convenient for simple plots, it is easy to run into problems.

For example, once the second panel is created, how can we go back and add something to the first? This interface does have a solution but it is not a easy solution.

Object-oriented interface of Matplotlib

The object-oriented interface can nicely handle more complicated plots, and for when you want more control over your figure.

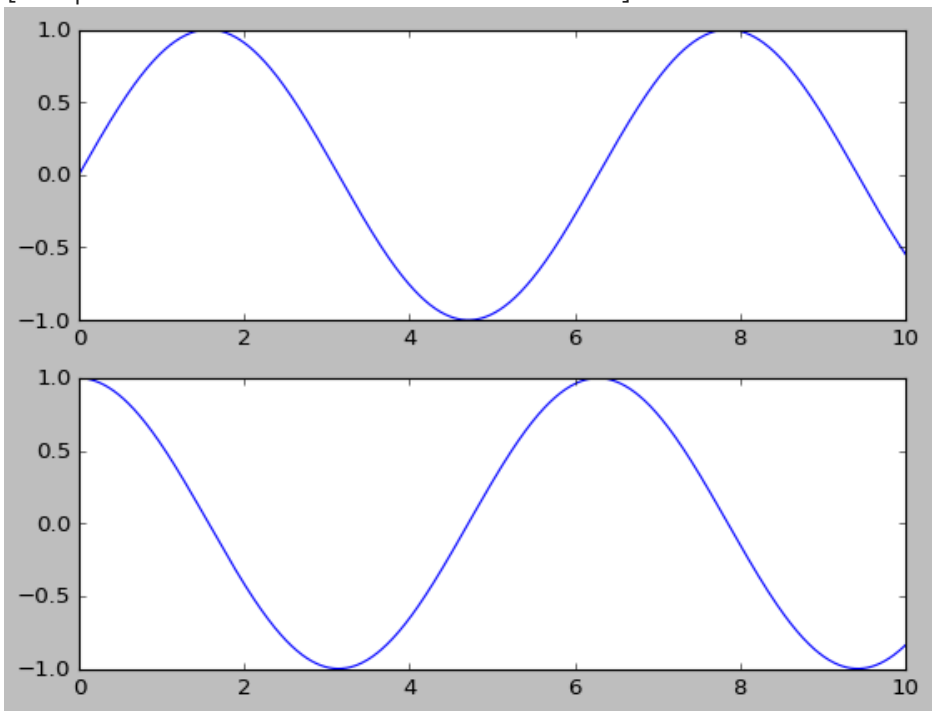
Rather than depending on some notion of an "active" figure or axes, in the object-oriented interface the plotting functions are *methods* of explicit `Figure` and `Axes` objects.

To re-create the previous plot using this style of plotting, you might do the following:

```
#First create a grid of plots
# ax will be an array of two Axes objects
fig, ax = plt.subplots(2)

# Call plot() method on the appropriate object
ax[0].plot(x, np.sin(x))
ax[1].plot(x, np.cos(x))
```

```
[<matplotlib.lines.Line2D at 0x7a3be036cf50>]
```



The difference is as small as switching `plt.plot()` to `ax.plot()`.

Now we start exploring different types of plots in following sections.

Simple Line Plots

Perhaps the simplest of all plots is the visualization of a single function `y = f(x)`.

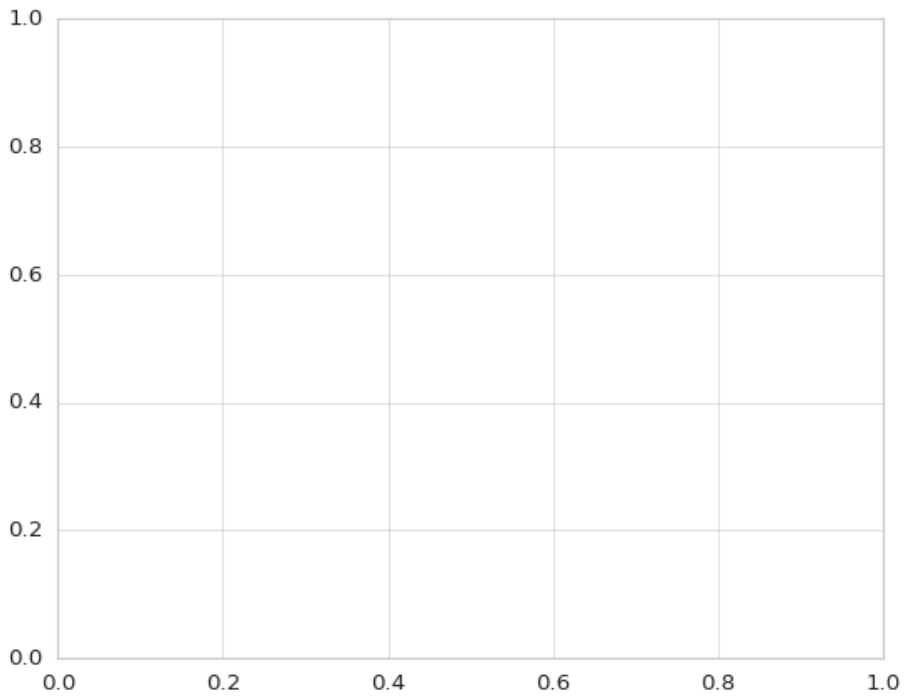
Here we will take a first look at creating a simple plot of this type.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns # Import seaborn

%matplotlib inline

# Plotting style chosen is that of seaborn-whitegrid
sns.set_style('whitegrid')
```

```
# creating objects of figure and axes.
fig = plt.figure()
ax = plt.axes()
```

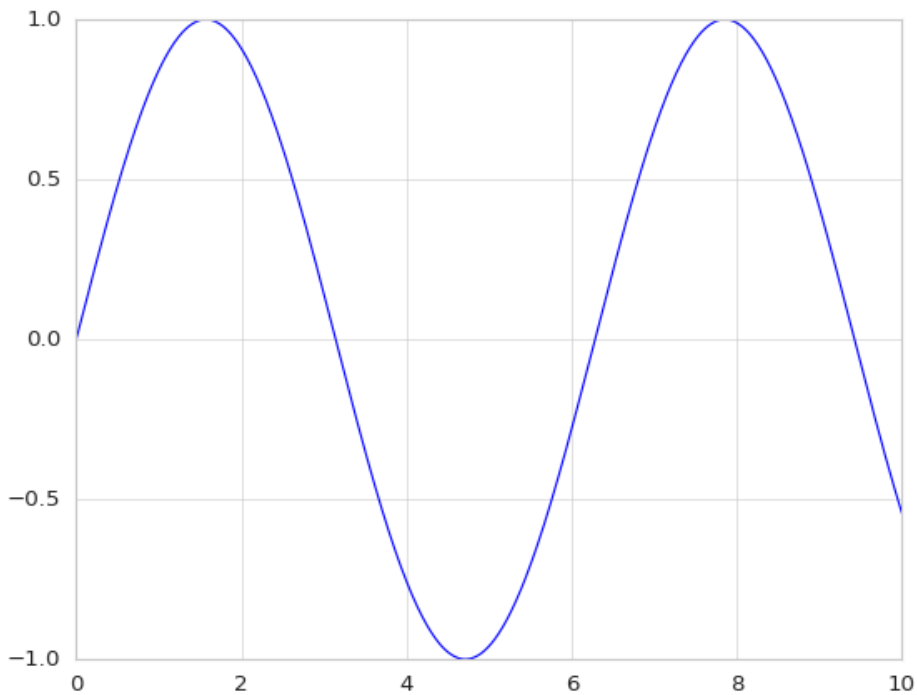


In Matplotlib, the *figure* (an instance of the class `plt.Figure`) can be thought of as a single container that contains all the objects representing axes, graphics, text, and labels.

The *axes* (an instance of the class `plt.Axes`) is what we see above: a bounding box with ticks and labels, which will eventually contain the plot elements that make up our visualization.

Once we have created an axes, we can use the `ax.plot` function to plot some data.

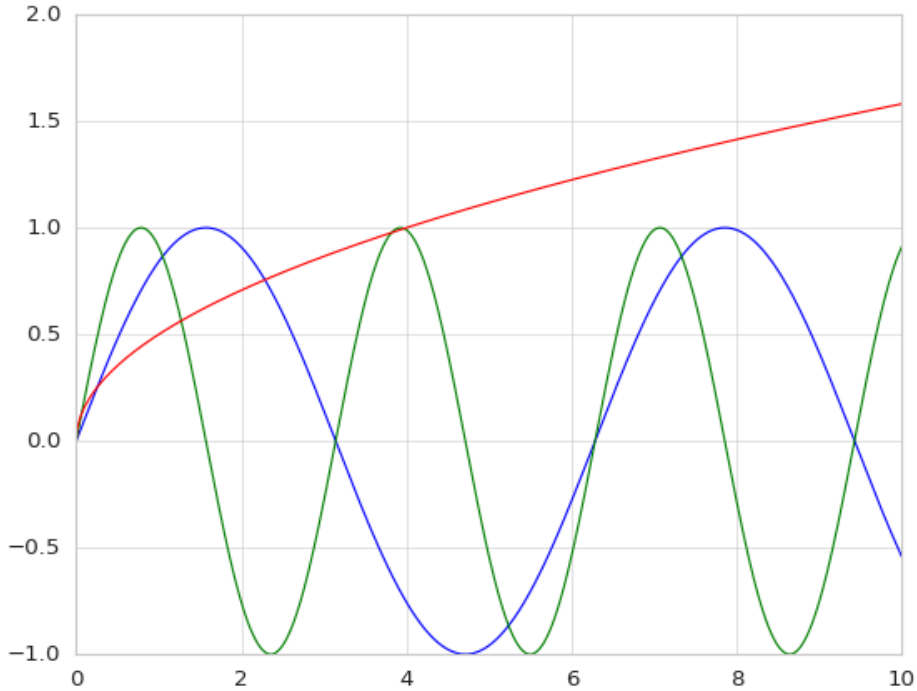
```
# we will plot sine curve using ax.plot function
fig = plt.figure()
ax = plt.axes()
x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
```



The word line in line plot refers to the function we want to plot. Multiple functions can be plot on same plot like shown below.

```
plt.plot(x, np.sin(x))   #shown in blue
plt.plot(x, np.sin(2*x)) #Shown in green
plt.plot(x,(x**(1/2))/2) #Shown in red
```

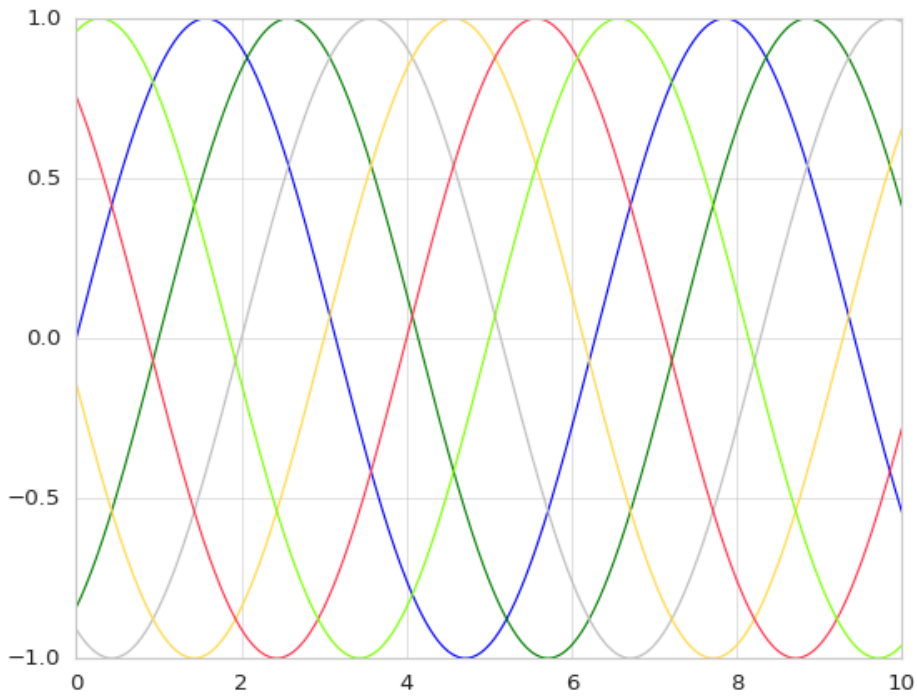
```
[<matplotlib.lines.Line2D at 0x7a3be295a180>]
```



Adjusting the Plot: Line Colors and Styles

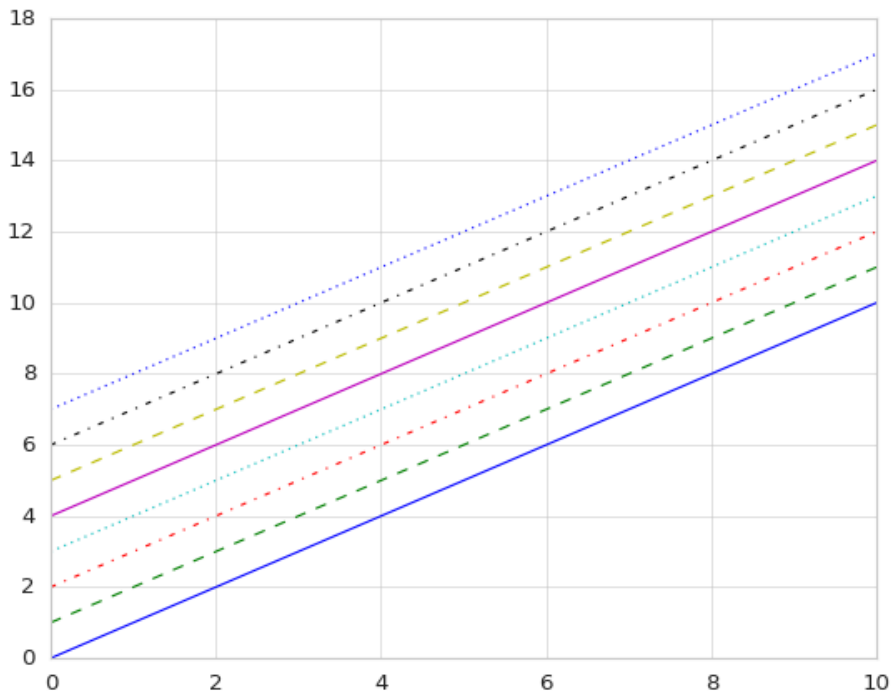
The first adjustment you might wish to make to a plot is to control the line colors and styles. The *plt.plot()* function takes additional arguments that can be used to specify these. To adjust the color, you can use the color keyword, which accepts a *string* argument representing virtually any imaginable color. The color can be specified in a variety of ways:

```
plt.plot(x, np.sin(x - 0), color='blue')        # specify color by name
plt.plot(x, np.sin(x - 1), color='g')           # short color code (rgbcmyk)
plt.plot(x, np.sin(x - 2), color='0.75')        # Grayscale between 0 and 1
plt.plot(x, np.sin(x - 3), color='#FFDD44')     # Hex code (RRGGBB from 00 to FF)
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 to 1
plt.plot(x, np.sin(x - 5), color='chartreuse'); # all HTML color names supported
```




```
plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

# For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-')  # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':');  # dotted
```

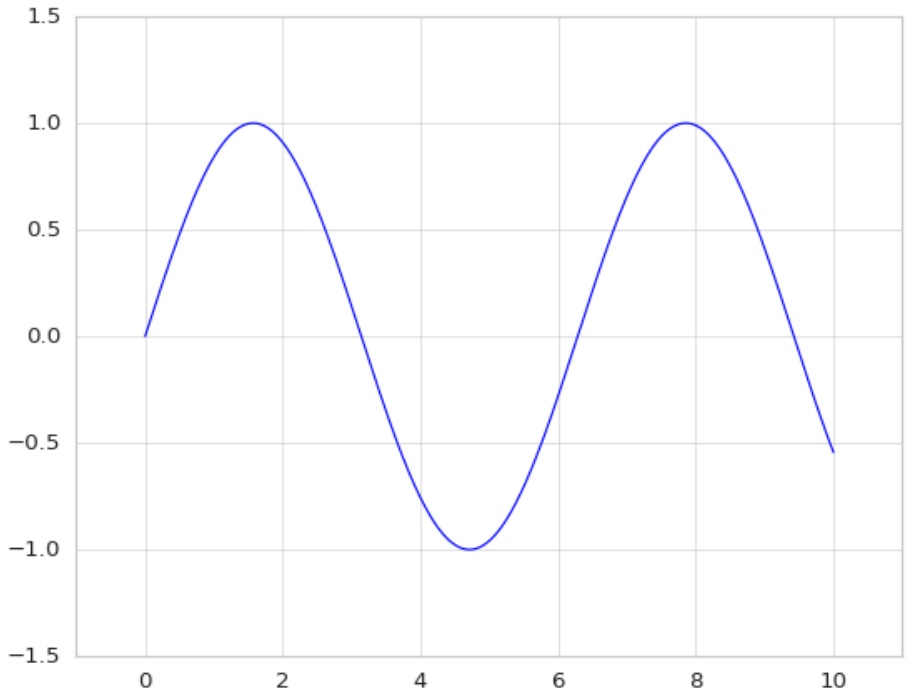


Adjusting the Plot: Axes Limits

Matplotlib does a decent job of choosing default axes limits for your plot, but sometimes it's nice to have finer control. The most basic way to adjust axis limits is to use the *plt.xlim()* and *plt.ylim()* methods:

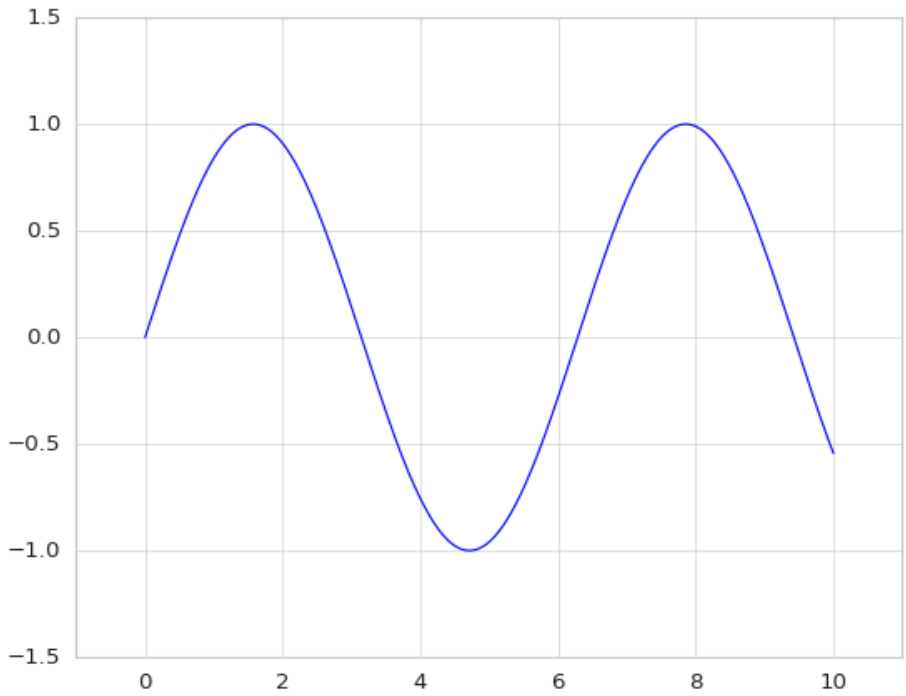
```
plt.plot(x, np.sin(x))

plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5);
```



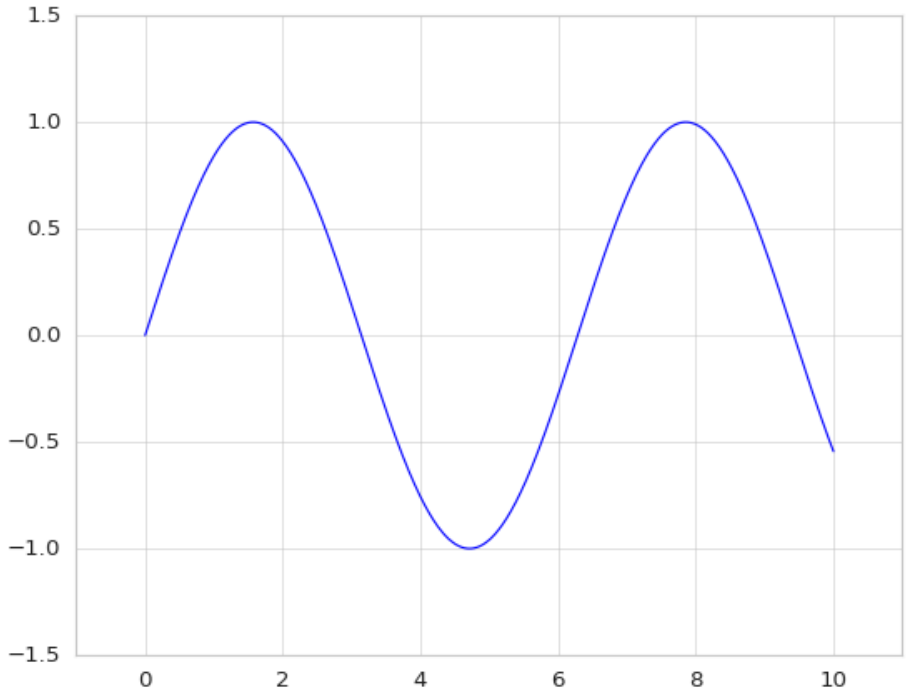

```
plt.plot(x, np.sin(x))

plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5);
```



A useful related method is plt.axis() (note here the potential confusion between axes with an e, and axis with an i). The plt.axis() method allows you to set the x and y limits with a single call, by passing a list which specifies **[xmin, xmax, ymin, ymax] :**

```
plt.plot(x, np.sin(x))
plt.axis([-1, 11, -1.5, 1.5]);
```

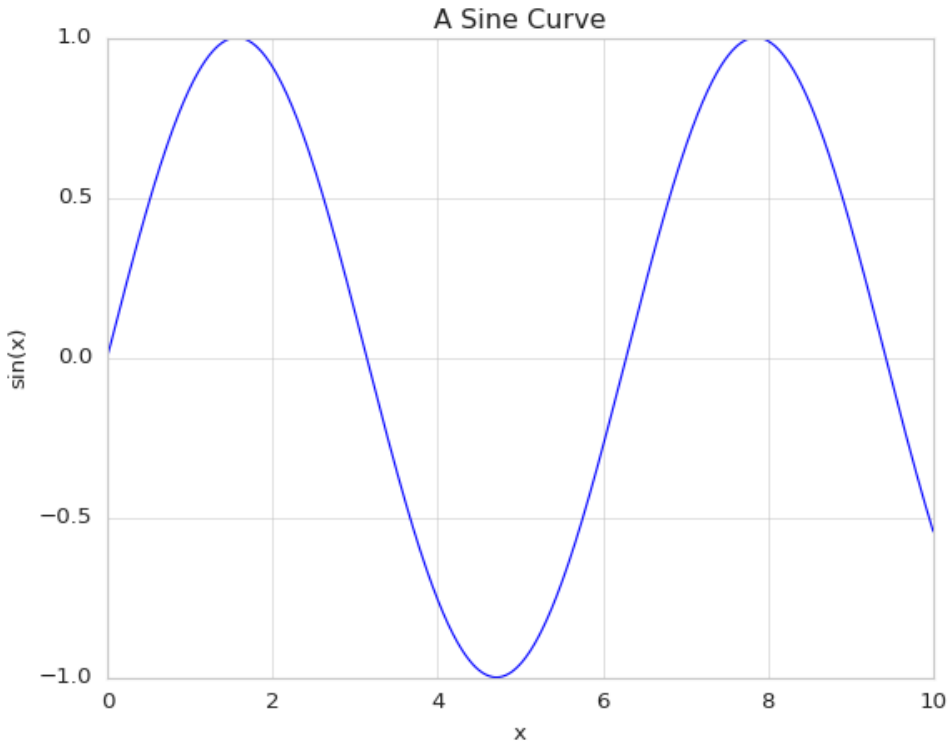


Labeling Plots

As the last piece of this section, we'll briefly look at the labeling of plots: titles, axis labels, and simple legends.

Titles and axis labels are the simplest such labels—there are methods that can be used to quickly set them:

```
plt.plot(x, np.sin(x))
plt.title("A Sine Curve")
plt.xlabel("x")
plt.ylabel("sin(x)");
```



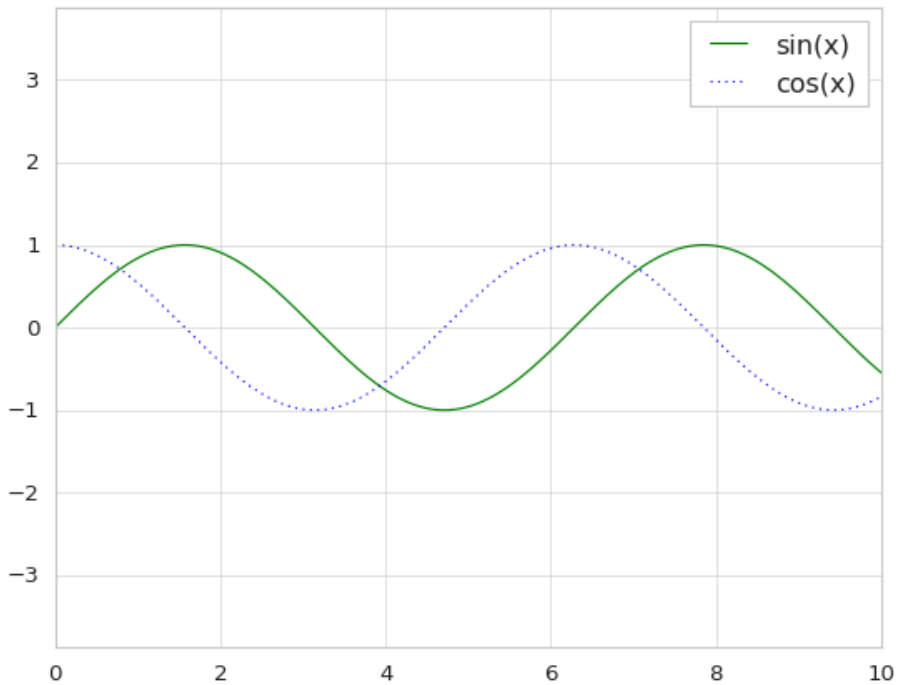
When multiple lines are being shown within a single axes, it can be useful to create a plot legend that labels each line type.

Again, Matplotlib has a built-in way of quickly creating such a legend. It is done via the (you guessed it) **plt.legend()** method.

Though there are several valid ways of using this, I find it easiest to specify the label of each line using the label keyword of the plot function:

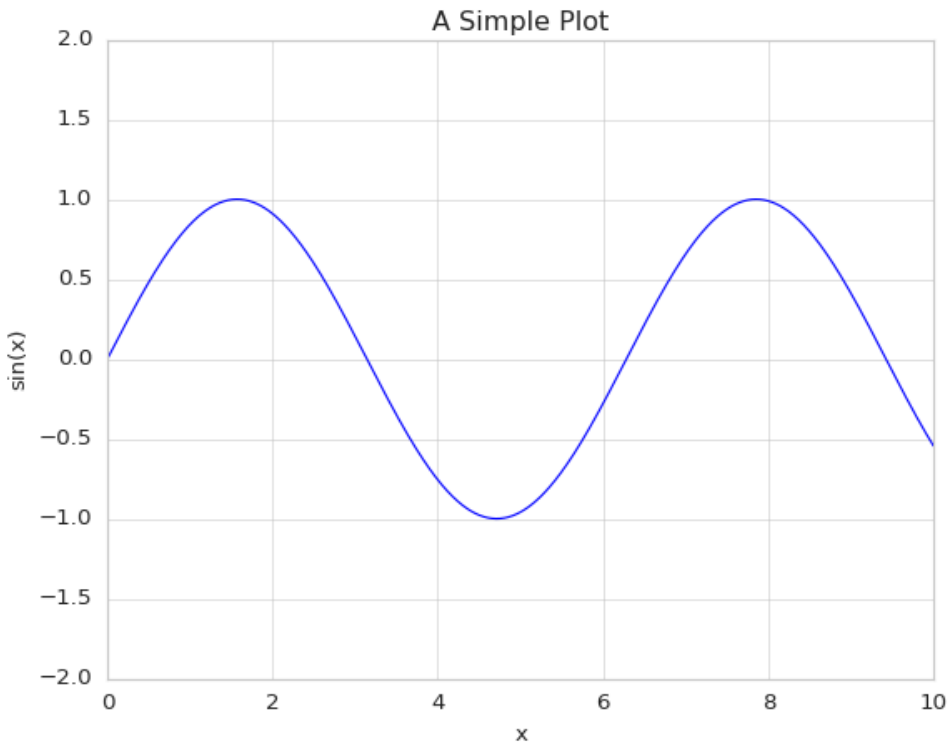
```
plt.plot(x, np.sin(x), '-g', label='sin(x)')
plt.plot(x, np.cos(x), ':b', label='cos(x)')
plt.axis('equal')

plt.legend();
```



In the object-oriented interface to plotting, rather than calling these functions individually, it is often more convenient to use the ax.set() method to set all these properties at once:

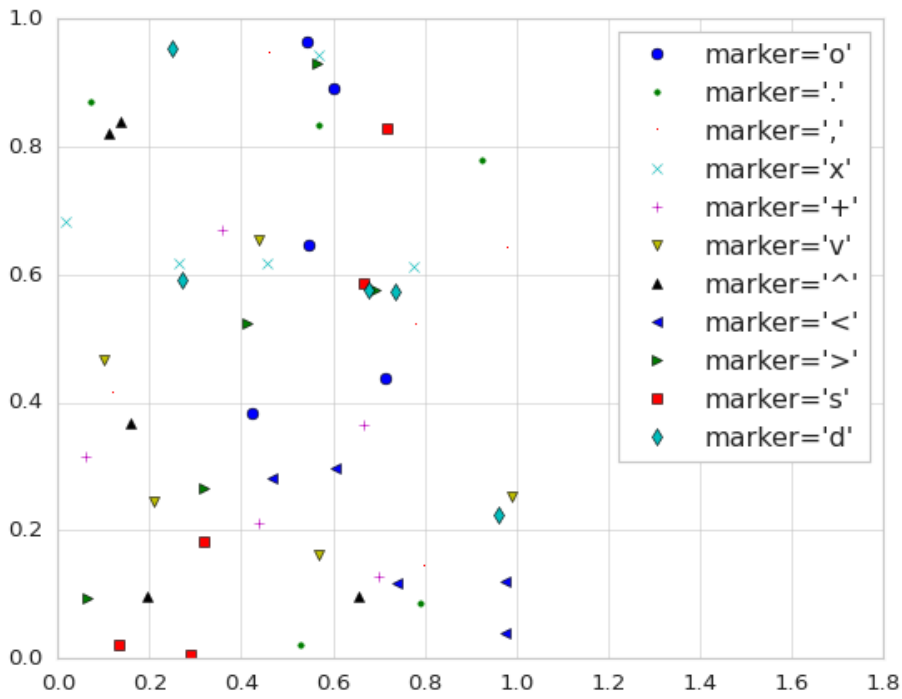
```
ax = plt.axes()
ax.plot(x, np.sin(x))
ax.set(xlim=(0, 10), ylim=(-2, 2),
       xlabel='x', ylabel='sin(x)',
       title='A Simple Plot');
```



Simple Scatter Plots

Another commonly used plot type is the simple scatter plot, a close cousin of the line plot. Instead of points being joined by line segments, here the points are represented individually with a dot, circle, or other shape.

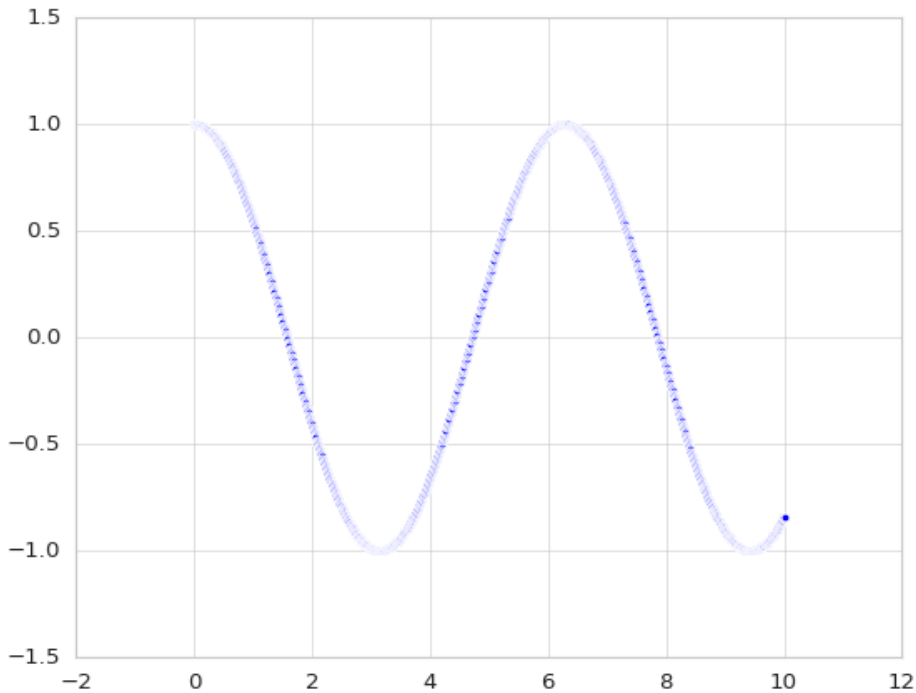
```
rng = np.random.RandomState(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8);
```



Scatter Plots with plt.scatter

A second, more powerful method of creating scatter plots is the plt.scatter function, which can be used very similarly to the plt.plot function:

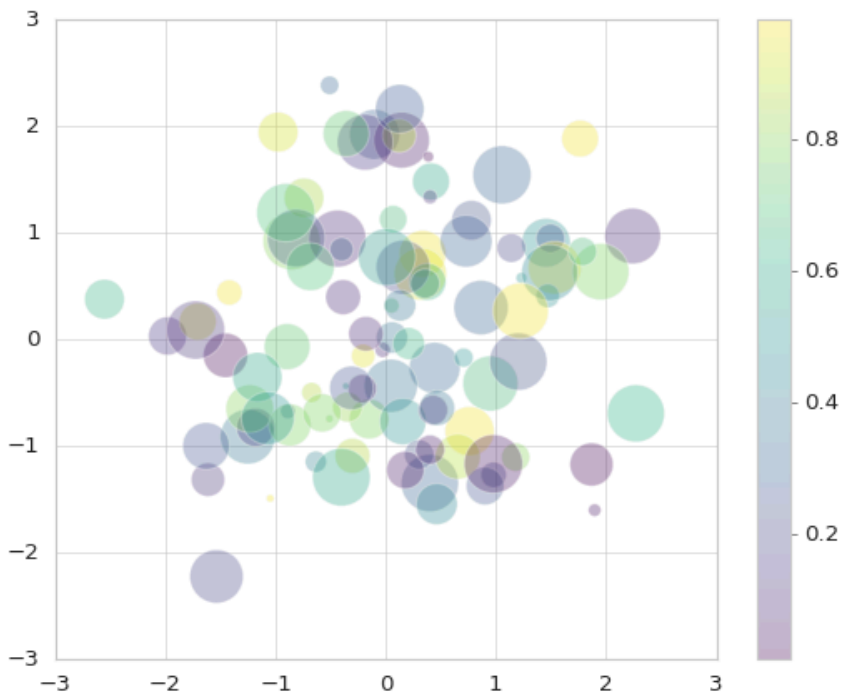
```
y = np.cos(x)
plt.scatter(x, y, marker='o');
```




```
# Consider this code for scatter plot

rng = np.random.RandomState(0)
x = rng.randn(100)
y = rng.randn(100)
colors = rng.rand(100)
sizes = 1000 * rng.rand(100)
```

```
plt.scatter(x, y, c=colors, s=sizes, alpha=0.3,
            cmap='viridis')
plt.colorbar();  # show color scale
```



Question Answer Section

1.List any 2 python APIs that are used for Plotting Graphs.

Ans: Two Python APIs used for plotting graphs are:

Matplotlib:

It is a basic and widely used plotting library in Python. It allows creation of line plots, bar charts, scatter plots, histograms, and more with full control.

Seaborn:

It is built on top of Matplotlib and provides more attractive and statistical graphics. It is mainly used for visualizing distributions and relationships in data easily.

2. Name 2 important machnisms used for accesssing Matplotlib API.

Ans: Two important mechanisms used for accessing the Matplotlib API are:

1. **Pyplot Interface (matplotlib.pyplot)** It provides simple and quick plotting functions similar to MATLAB style. It is easy to use for basic and quick graphs.
2. **Object-Oriented (OO) Interface** It uses `Figure` and `Axes` objects to create plots. It gives more control and flexibility for complex and multiple plots.

3. Create a collection of 100 random normal numbers with parameter mean 30, sd 4 store it in object summer_temp

Ans: To create a collection of 100 random normal numbers with mean 30 and standard deviation 4, we use NumPy.

```
import numpy as np

summer_temp = np.random.normal(loc=30, scale=4, size=100)
```

Here,

- `loc=30` represents the mean,
- `scale=4` represents the standard deviation,
- `size=100` generates 100 random values.

The generated values are stored in the object `summer_temp`.

4. Sort above set and store in objrct sorted_summer_temp

Ans: To sort the previously created `summer_temp` data and store it in a new object:

```
sorted_summer_temp = np.sort(summer_temp)
```

Here, `np.sort()` arranges the values in ascending order. The sorted values are stored in the object `sorted_summer_temp`.

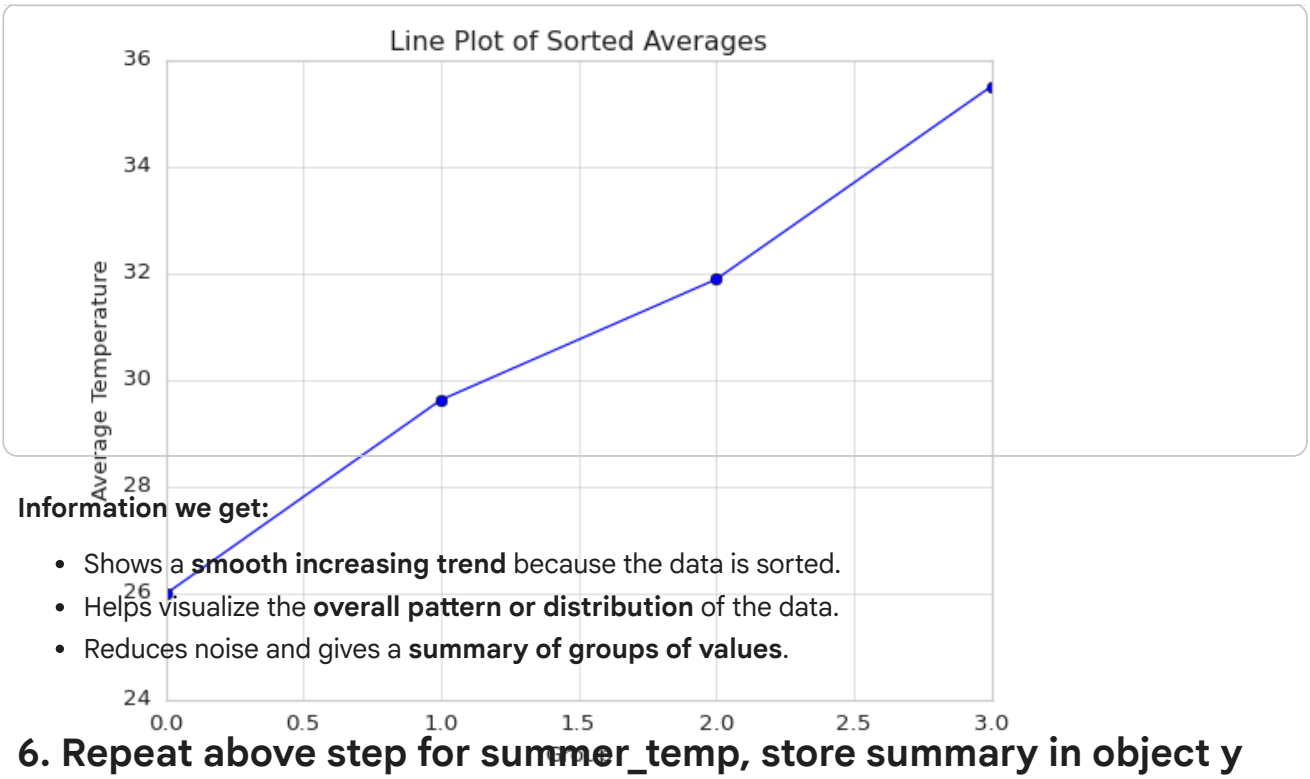
5. Store average of 25 consecutive elements from sorted_summer_temp into object x and plot it as a line plot. What kind of information do u get about the data?

Ans:Here's how to calculate the average of 25 consecutive elements from `sorted_summer_temp` and plot it:

```
import matplotlib.pyplot as plt

# Calculate average of every 25 consecutive elements
x = [np.mean(sorted_summer_temp[i:i+25]) for i in range(0, 100, 25)]

# Plot as a line plot
plt.plot(x, marker='o')
plt.title("Line Plot of Sorted Averages")
plt.xlabel("Group")
plt.ylabel("Average Temperature")
plt.show()
```



Information we get:

- Shows a **smooth increasing trend** because the data is sorted.
- Helps visualize the **overall pattern or distribution** of the data.
- Reduces noise and gives a **summary of groups of values**.

6. Repeat above step for summer_temp, store summary in object y and this time use scatter plot to plot values. What kind of information do u get about the data?

Ans: Here's a clear and precise way to calculate and plot the averages for the unsorted data:

```
# Compute average of every 25 consecutive elements from unsorted data
y = [np.mean(summer_temp[i:i+25]) for i in range(0, 100, 25)]

# Scatter plot of the averages
plt.scatter(range(len(y)), y, color='red')
plt.title("Scatter Plot of Unsorted Averages")
plt.xlabel("Group Index")
plt.ylabel("Average Value")
plt.show()
```